Generalization

Trắc nghiệm 29 Thực hành 3 PDF bài

> Sau buổi đầu, một thành viên đề xuất lưu nguyên bảng các buổi sinh hoạt: gặp lại đăng ký và dự báo mưa cũ thì trả đúng số người đã đến. Trên dữ liệu đã lưu, sai số có thể bằng 0. Nhưng thứ Bảy tuần sau là một buổi mới. Nếu thông tin hơi khác, bảng nhớ ấy sẽ đoán thế nào?

Khả năng dự đoán tốt trên mẫu chưa dùng để học gọi là khả năng tổng quát hóa (generalization). Học máy không kết thúc ở việc giảm sai số train. Bài này dùng một mô hình dễ hiểu để nhìn thấy underfitting, overfitting và ý nghĩa của learning curve, rồi quay lại quyết định nên thử gì tiếp.

SAU BÀI NÀY, CÓ THỂ

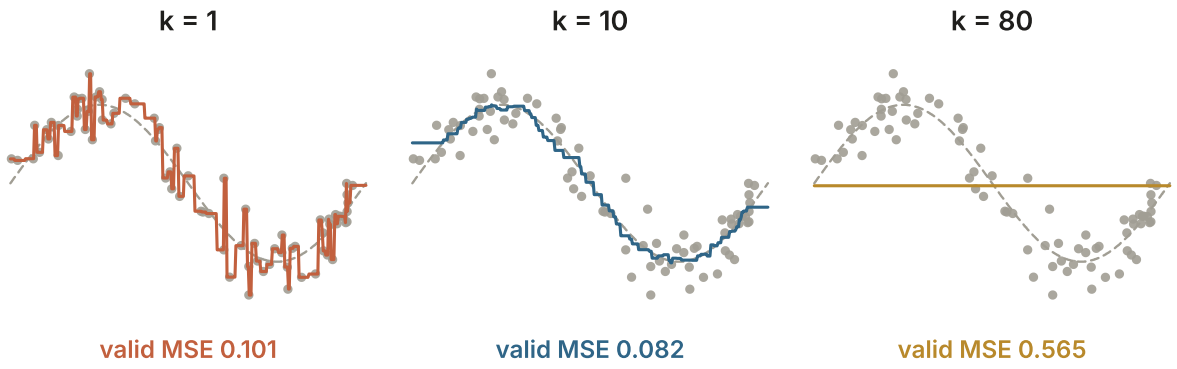
- Phân biệt lỗi train với lỗi trên dữ liệu mới và đọc khoảng cách giữa chúng.
- Nhận diện bằng chứng của underfit, overfit qua một thí nghiệm có kiểm soát.
- Đọc learning curve theo số mẫu, phân biệt với đường loss theo vòng huấn luyện.
- Chọn bước xử lý dựa trên dữ liệu, đồng thời kiểm tra leakage và thay đổi phân phối.

Nhớ một ví dụ khác với học một quy luật

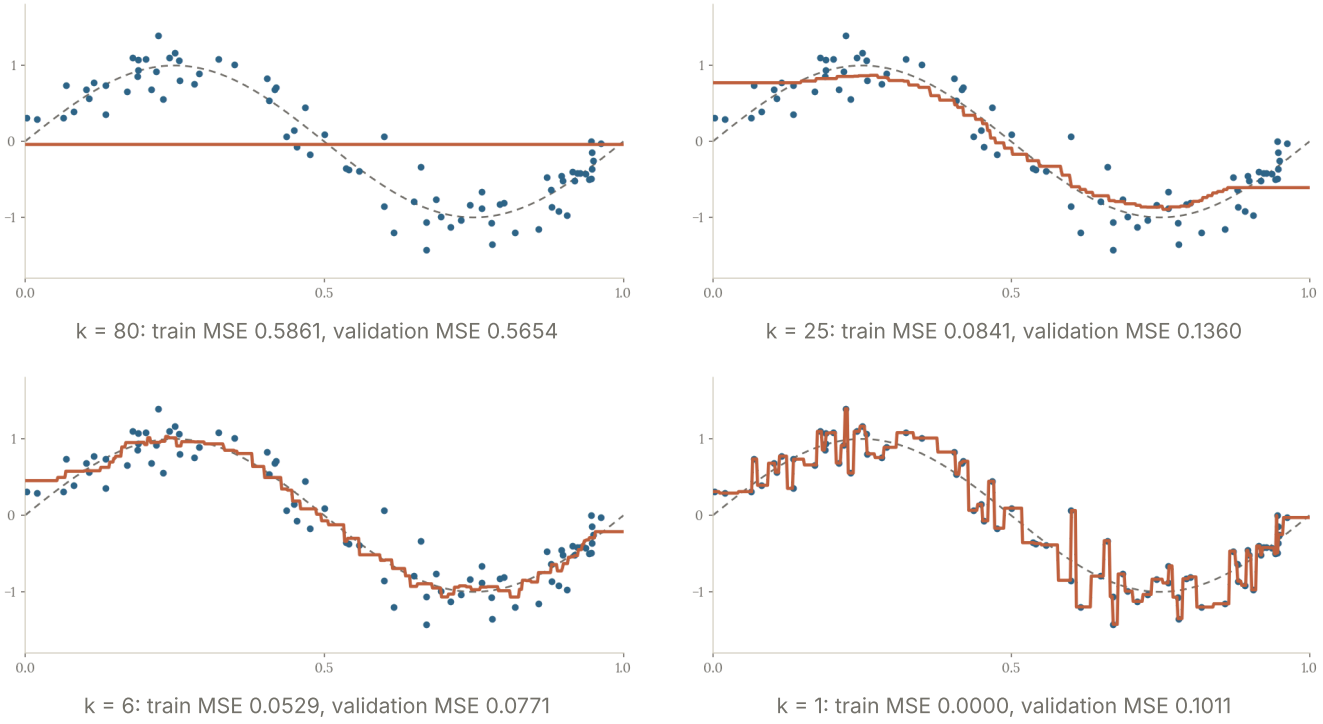
Ta tạm thu bài toán xuống một feature x trong khoảng 0–1: vị trí của một buổi trong lịch hoạt động. Mức tham gia đã chuẩn hóa là

$$y = \sin(2\pi x) + \varepsilon.$$

sin tạo một xu hướng lên rồi xuống để nhìn được bằng hình. ε là nhiễu, ở đây sinh ngẫu nhiên với trung bình 0 và độ lệch chuẩn 0.25. Các giá trị âm có nghĩa là thấp hơn mức tham gia chuẩn, không phải số người âm. Dạng sóng này do ta tự đặt ra để thí nghiệm, nhờ đó biết chính xác đâu là quy luật, đâu là nhiễu. Học được là nhờ có quy luật: nếu nhãn hoàn toàn ngẫu nhiên như kết quả xổ số thì không mô hình nào đoán được, dù dữ liệu nhiều đến đâu.



k = 1 bám nhiễu, k = 80 san phẳng, k = 10 vừa đủ.



kNN hồi quy trên 80 điểm train của bài khi giảm k từ 80 về 1: đường cam đi từ quá phẳng tới bám từng điểm nhiễu; nét đứt là quy luật thật.

Mô hình trong hình là k-Nearest Neighbors regression, viết ngắn là kNN. Để đoán tại x, nó tìm k điểm train gần nhất rồi lấy trung bình nhãn của chúng. Bài 2.4 sẽ học thuật toán kỹ hơn; hôm nay chỉ dùng đặc tính: k nhỏ cho đường linh hoạt hơn, k lớn cho đường trơn hơn.

Với $k = 1$, tại một điểm train, hàng xóm gần nhất chính là nó, nên mô hình trả đúng nhãn đã quan sát. Điều ấy bao gồm cả phần nhiễu của riêng điểm đó. Với k bằng toàn bộ số mẫu train, mọi vị trí nhận cùng một giá trị trung bình, dù xu hướng thật lên xuống rõ rệt.

HỎI NHANH

Mô hình đoán đúng mọi nhãn trên tập train. Điều đó chứng minh được gì?

A Nó khớp các mẫu đã học, còn mẫu mới thì chưa biết

B Nó chắc chắn đã tìm ra quy luật thật

C Dữ liệu train chắc chắn không có nhãn nhiễu

D Nó sẽ đúng ít nhất 90% trên dữ liệu mới

Viết rõ đang đo lỗi ở đâu

Giả sử có n mẫu train. Sai số train với MSE là

$$\widehat{R}_{\text{train}}(f) = \frac{1}{n}\sum_{i=1}^{n}(f(x_i) - y_i)^2.$$

Ký hiệu $\widehat{R}$ nhắc rằng đây là một phép ước lượng từ hữu hạn mẫu. Rủi ro trên mẫu mới có thể viết

$$R(f) = \mathbb{E}_{(x,y)\sim P}\left[(f(x) - y)^2\right].$$

$\mathbb{E}$ nghĩa là trung bình nếu ta có rất nhiều mẫu mới; P là cách sinh dữ liệu trong tình huống sử dụng. Ta không biết toàn bộ P, nên dùng validation được chia phù hợp để ước lượng lỗi. Điều kiện quan trọng là validation phải mô phỏng được dữ liệu mới; một split sai có thể cho điểm đẹp nhưng trả lời câu hỏi khác. Khả năng làm tốt trên mẫu chưa gặp lúc học gọi là khái quát hóa (generalization); đó là thứ ta cần, không phải điểm trên train.

Trong thí nghiệm này, train và validation được sinh độc lập từ cùng công thức. Vì vậy ta có thể tập trung vào độ linh hoạt của mô hình. Code tạo 80 mẫu train và 160 mẫu validation, không dùng test để chọn k.

```
Python
```

```python
import numpy as np
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(12)

def make_data(n):
    X = rng.uniform(0, 1, size=(n, 1))
    y = np.sin(2 * np.pi * X[:, 0]) + rng.normal(0, 0.25, size=n)
    return X, y

X_train, y_train = make_data(80)
X_valid, y_valid = make_data(160)
print(X_train.shape, y_train.shape)
for k in [80, 10, 1]:
    model = KNeighborsRegressor(n_neighbors=k).fit(X_train, y_train)
    train_mse = mean_squared_error(y_train, model.predict(X_train))
    valid_mse = mean_squared_error(y_valid, model.predict(X_valid))
    print(f"k={k:2d}: train={train_mse:.4f}, valid={valid_mse:.4f}")
```

KẾT QUẢ

```
(80, 1) (80,)
k=80: train=0.5861, valid=0.5654
k=10: train=0.0574, valid=0.0820
k= 1: train=0.0000, valid=0.1011
```

Đọc cả hai cột. Khi $k = 80$, train và validation cùng có lỗi lớn: mô hình quá đơn giản để giữ xu hướng. Khi $k = 1$, train có lỗi bằng 0, nhưng validation không vì thế thấp nhất. $k = 10$ lấy trung bình cục bộ, giảm tác động của nhiễu mà còn giữ được dạng sóng.

Không chọn mô hình từ cột train. Cũng không biến ví dụ thành quy tắc "luôn dùng $k = 10$": số mẫu, mật độ điểm, độ nhiễu và số chiều thay đổi thì k phù hợp cũng đổi.

HỎI NHANH

Phương án A có MSE 0.01 trên train và 0.30 trên validation. Phương án B có MSE 0.08 trên train và 0.12 trên validation. Chọn theo validation thì ta chọn phương án nào?

A A, vì MSE trên train nhỏ hơn

B A, vì chênh lệch lớn cho thấy nó học được nhiều hơn

C B, vì MSE trên validation nhỏ hơn

D Chưa chọn được, phải xem điểm test của cả hai

Underfit và overfit là chẩn đoán có điều kiện

Underfitting (học chưa đủ) là tình huống mô hình không biểu diễn hoặc không học được cấu trúc cần thiết. Trong ví dụ, $k = 80$ san phẳng đường cong. Thêm dữ liệu từ cùng phân phối vẫn không khiến phương án đoán hằng số biến thành đường sóng. Ta có thể cần mô hình linh hoạt hơn hoặc feature tốt hơn.

Overfitting (học quá sát) là tình huống mô hình khai thác những chi tiết của train mà không ổn định trên mẫu mới. Với $k = 1$, đổi một nhãn nhiễu có thể kéo một đoạn dự đoán theo nó. Để giảm điều ấy, có thể tăng k, dùng regularization ở chương 2, hoặc tăng dữ liệu có ích.

Dấu hiệu thường gặp: train tốt mà validation kém hẳn là overfitting; train và validation cùng kém, gần nhau, là underfitting. Overfitting dễ xảy ra khi mô hình có rất nhiều tham số mà dữ liệu ít, hay có nhiều đặc trưng hơn số mẫu, như 100 chỉ số xét nghiệm cho 50 bệnh nhân. Một mạng đủ lớn còn học thuộc được cả nhãn bị xáo ngẫu nhiên, đạt 100% trên train, nên điểm train cao chưa chứng minh mô hình học được quy luật. Underfitting cần mô hình mạnh hơn hoặc đặc trưng tốt hơn; giảm regularization hay thêm dữ liệu cùng loại không đủ, vì mô hình chưa khớp nổi cả train.

Hai tên này là giả thuyết, không phải kết luận. Train tốt mà validation tệ cũng có thể do split thời gian chứa một đợt thay đổi lớn, cột feature dùng sai đơn vị, hoặc pipeline biến đổi hai tập khác nhau. Train và validation cùng tệ cũng có thể do nhãn bị đọc lệch hàng hay metric cài sai. Trước khi sửa mô hình, kiểm hợp đồng dữ liệu và một ví dụ nhỏ.

HỎI NHANH

Một mô hình phân loại review phim là khen hay chê, có 2.4 triệu tham số, học trên 5000 review. Nó đoán đúng 98.5% trên train, 72.3% trên validation và 71.8% trên test. Số liệu này chỉ ra vấn đề gì?

A Lộ dữ liệu, vì validation và test gần bằng nhau

B Underfitting, vì 72% là thấp với bài này

C Overfitting, vì train gần hoàn hảo còn validation kém xa

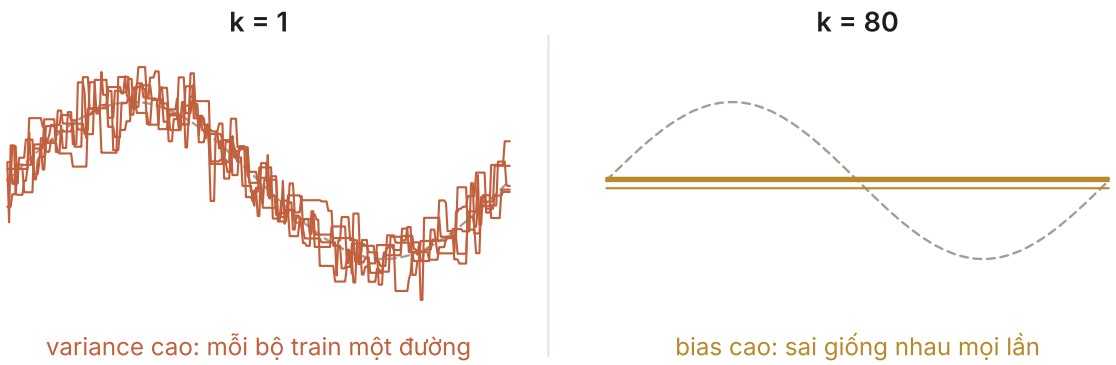
D Dữ liệu thay đổi, vì train cao còn test thấp

ĐỀ THI NOAI 2026 Singapore · Vòng sơ loại · Câu 272

BIAS VÀ VARIANCE Ở MỨC TRỰC GIÁC

Bias là khuynh hướng sai có hệ thống do giả định mô hình, chẳng hạn ép mọi x về cùng một dự đoán. Variance là mức mô hình thay đổi khi ta thay bộ train. Mô hình linh hoạt thường giảm được bias nhưng có thể tăng variance. Không thể suy ra chính xác hai đại lượng này chỉ từ một cặp điểm train/validation; cần nhìn nhiều lần lấy mẫu hoặc phép đánh giá thích hợp.

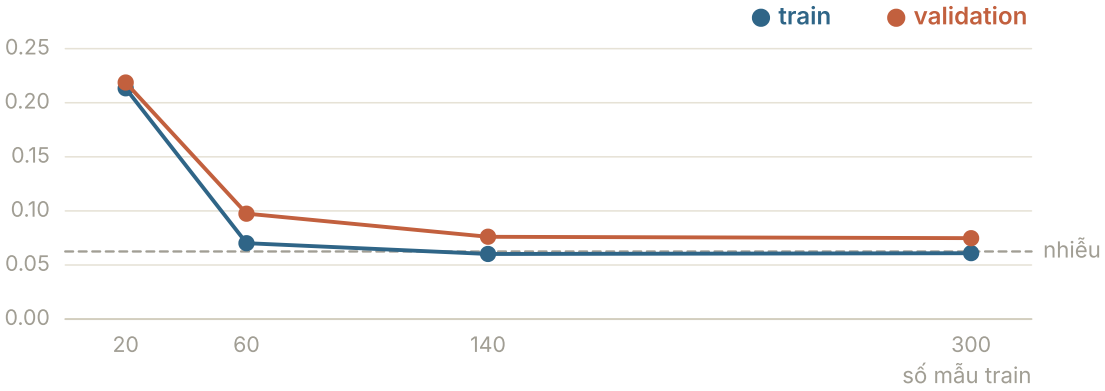
Với MSE, tại một điểm x, lỗi kỳ vọng tách thành ba phần: $\mathbb{E}[(y - \hat{f}(x))^2] = \sigma^2 + \text{bias}^2 + \text{variance}$, trong đó bias là độ lệch giữa dự đoán trung bình (trên nhiều bộ train) và giá trị thật, variance là phương sai của dự đoán quanh trung bình ấy. σ^2 là phương sai của nhiễu, sai số không thể giảm (irreducible error): không mô hình nào đoán được phần nhiễu. Bias và variance thường đánh đổi nhau khi chỉnh độ phức tạp, nhưng không có luật nào buộc chúng ngược chiều: đoán bừa có cả bias lẫn variance cao.



Năm bộ train: k = 1 đổi theo từng bộ, k = 80 lần nào cũng sai như nhau.

Learning curve: thêm dữ liệu có đáng làm không?

Ta giữ $k = 10$ và tăng số mẫu được học. Với mỗi mức, learning curve đo lỗi train và lỗi validation. Trục ngang là **số mẫu**, không phải epoch. Một đường loss trong quá trình tối ưu có trục ngang là số bước học và trả lời câu hỏi khác.



Thêm mẫu, hai đường lỗi gần nhau và sát mức nhiễu.

Code sau dùng `learning_curve`; nó học những bản mô hình mới cho từng kích thước và từng fold. Fold là một lần giữ riêng một phần dữ liệu để chấm, sẽ học kỹ ở bài 1.3. Không có nhãn validation đi vào `fit` của lần học tương ứng.

```python
from sklearn.model_selection import KFold, learning_curve

X_curve, y_curve = make_data(400)
cv = KFold(n_splits=4, shuffle=True, random_state=12)
```

```python
sizes, train_scores, valid_scores = learning_curve(
    KNeighborsRegressor(n_neighbors=10),
    X_curve, y_curve, cv=cv,
    train_sizes=[20, 60, 140, 300],
    scoring="neg_mean_squared_error",
    shuffle=True, random_state=12, n_jobs=1,
)
for size, train_score, valid_score in zip(
    sizes, train_scores.mean(axis=1), valid_scores.mean(axis=1)
):
    print(f"n={size:3d}: train MSE={-train_score:.4f}, valid MSE={-valid_score:.4f}")
```

KẾT QUẢ

```
n= 20: train MSE=0.2133, valid MSE=0.2187
n= 60: train MSE=0.0701, valid MSE=0.0974
n=140: train MSE=0.0602, valid MSE=0.0761
n=300: train MSE=0.0610, valid MSE=0.0748
```

Các score có dấu âm vì công cụ chọn mô hình của scikit-learn quy ước "lớn hơn là tốt hơn", nên MSE được đổi dấu thành neg-MSE; ta đổi dấu lại để đọc. Mỗi hàng là trung bình của bốn fold.

Với ít mẫu, mười hàng xóm phủ một vùng rộng, làm trơn quá mức. Thêm mẫu khiến vùng đó hẹp hơn và giữ được xu hướng. Đến lúc đường validation giảm rất chậm, thêm vài mẫu có thể chỉ cải thiện ít; cần kiểm feature, độ nhiễu và mô hình thay vì mặc định cứ thu thập thêm là đủ. Hai đường gặp nhau rồi cùng đi ngang ở mức lỗi cao là underfitting: thêm dữ liệu không giúp, cần mô hình mạnh hơn. Khoảng cách lớn giữa hai đường mà đường validation còn giảm thì thêm dữ liệu có ích; đường validation đã đi ngang thì nên giảm độ phức tạp, tăng regularization hay bỏ bớt đặc trưng. Đổi tỉ lệ chia cũng vậy: thêm train thường có ích nhưng không chắc, còn tập đánh giá nhỏ đi thì con số đo được dao động nhiều hơn.

Có một mốc để so: kể cả khi biết chính xác $\sin(2\pi x)$, lỗi kỳ vọng vẫn còn bằng phương sai nhiễu $0.25^2 = 0.0625$, vì không mô hình nào đoán được nhiễu. Validation MSE 0.0748 ở $n = 300$ đã khá gần mốc ấy, nên dư địa cải thiện còn ít. (Trên một tập validation hữu hạn, lỗi đo được có thể thấp hơn 0.0625 đôi chút do ngẫu nhiên.) Với dữ liệu thật ta không biết mức nhiễu; kết quả tốt nhất mà con người đạt được, như một nhóm bác sĩ hội chẩn, thường được dùng làm ước lượng cho mức lỗi thấp nhất có thể (sai số Bayes).

HỎI NHANH

Tăng train từ 200 lên 800 rồi 2000 mẫu. MSE trên train lần lượt là 0.42, 0.43, 0.43. MSE trên validation là 0.46, 0.45, 0.45. Cách chia và metric đều đã kiểm. Ta nên thử gì trước?

A Giữ nguyên mọi thứ, chỉ thu thêm mẫu

B Thêm feature hoặc dùng mô hình mạnh hơn

C Giảm độ phức tạp để thu hẹp khoảng cách

D Đổi seed chia dữ liệu tới khi validation thấp hơn

PHỎNG THEO CMU 10-601 · Practice Problems, Exam 1 · Q3.1

Ví dụ tính tay

Giữ $k = 2$. Hai hàng xóm của một buổi mới có nhãn 1 và 3. Dự đoán là $(1 + 3)/2 = 2$. Nếu nhãn thật của buổi mới là 4, lỗi bình phương là $(2 - 4)^2 = 4$.

Trên ba buổi, nhãn thật $(1, 3, 5)$ và dự đoán $(2, 3, 4)$ cho MSE $= (1 + 0 + 1)/3 = 2/3$. Một phương án khác dự đoán đúng ba nhãn train chưa cho ta thêm thông tin nào về ba buổi này. Cần phân biệt rõ "những buổi nào" trong mỗi phép tính.

Khi so hai phương án trên nhiều fold, nên đọc từng fold và chênh lệch trên cùng fold. Một mô hình cải thiện ở mọi fold có bằng chứng ổn định hơn mô hình chỉ thắng nhờ một fold thuận lợi. Độ lệch chuẩn giữa các fold không tự là khoảng tin cậy vì các train của chúng có thể chồng nhau.

Trở lại câu chuyện câu lạc bộ

Mô hình học trên các buổi trong năm rồi dùng vào dịp hè có thể gặp distribution shift (thay đổi phân phối): số đăng ký, lịch học và thói quen tham gia cùng đổi. Nếu các buổi hè rất khác, một split ngẫu nhiên trộn cả hè vào train sẽ che bớt khó khăn đó. Mô hình chỉ đáng tin trong vùng dữ liệu đã học: học giá nhà 40–120 m^2 rồi đoán căn 600 m^2 là ngoại suy; nhận diện khuôn mặt học trên người lớn sẽ kém với trẻ em; ảnh chụp ban ngày không dạy được ảnh ban đêm; dữ liệu năm 2023 có thể đã khác năm 2025. Cách sửa gốc là đưa loại dữ liệu lúc dùng thật vào train và đánh giá trên dữ liệu gần thời điểm dùng.

Vậy generalization luôn gắn với một mục tiêu sử dụng. Dự đoán buổi tương lai của cùng câu lạc bộ khác với dự đoán câu lạc bộ ở trường mới. Bài 1.3 biến hai câu hỏi này thành split theo thời gian và split theo group. Không có một khoảng cách train–validation nào tự chứng nhận mọi tình huống sử dụng.

HỎI NHANH

Ta cần dự báo cho tháng sau. Chia dữ liệu ngẫu nhiên thì MAE là 2. Giữ riêng tháng cuối làm validation thì MAE là 5. Nên tin con số nào?

A Bỏ tháng cuối khỏi dữ liệu cho phép đánh giá ổn hơn

B Tin số 2, vì MAE thấp hơn là tốt hơn

C Lấy trung bình 3.5 làm MAE cuối cùng

D Tin số 5, rồi tìm hiểu vì sao tháng cuối khó hơn

Khi đi thi

- **Ghi cả train và validation** cho mỗi lần thử, kèm số mẫu và cách chia; nhìn một cột thì không chẩn đoán được.

- **Thua baseline thì kiểm dữ liệu trước**, chưa vội đổi mô hình.
- **Train tốt, validation kém:** giảm độ linh hoạt, thêm regularization, xem lỗi theo nhóm. **Cả hai cùng kém:** thêm feature hoặc mô hình mạnh hơn.
- **So cải thiện nhỏ trên cùng các fold**, không dựa vào một con số đã làm tròn.

Tóm tắt

- Generalization là hiệu quả trên mẫu mới trong tình huống sử dụng đã nêu.
- Train nhỏ chưa đủ; đọc cả lỗi validation và điều kiện split.
- Underfit và overfit gợi ý cách xử lý, nhưng cần loại lỗi dữ liệu và leakage.
- Learning curve thay số mẫu; đường tối ưu thay số bước. Hai trục ngang khác nhau.

Kiểm tra cuối bài

CÂU 1

Một mô hình khớp train rất tốt. Bằng chứng nào cho thấy nó cũng sẽ đoán tốt khi đem dùng thật?

A File mô hình nhỏ và huấn luyện nhanh

B Lỗi thấp trên dữ liệu mới, chưa dùng để học hay để chọn

C Lỗi train còn giảm tiếp khi tăng số tham số

D Chạy lại với cùng seed thì dự đoán trên train không đổi

CÂU 2

kNN regression dùng trọng số đều, có 80 mẫu train, và ta chọn $k = 80$. Hai điểm cần đoán khác nhau sẽ nhận dự đoán thế nào?

A Cả hai đều bằng trung bình của 80 nhãn train

B Khác nhau, vì khoảng cách tới 80 mẫu khác nhau

C Cả hai đều bằng median của 80 nhãn train

D Mỗi điểm nhận nhãn của mẫu gần nó nhất

CÂU 3

Bài phân loại văn bản 10 lớp, có 5000 mẫu. Ba mô hình cho tỉ lệ đúng trên train và validation như sau. A là một mô hình rất đơn giản: 68% và 67%. B là một mô hình rất lớn, 2 triệu tham số: 99% và 71%. C là mô hình vừa phải, có regularization: 85% và 83%. Chẩn đoán nào đúng?

A A underfit, cần mô hình mạnh hơn. B overfit, cần regularization. C cân bằng.

B A khớp tốt, nên giữ. B bias cao, cần train lâu hơn. C variance cao, nên bớt regularization.

C A variance cao, cần thêm dữ liệu. B khớp tốt, đem dùng được. C bias cao, cần tăng sức chứa.

D Cả ba đều do dữ liệu bẩn, phải làm sạch trước khi chỉnh mô hình.

ĐỀ THI NOAI 2026 Singapore · Vòng sơ loại · Câu 277

CÂU 4

kNN regression với $k = 1$, chấm trên chính tập train. Khi nào thì lỗi train chắc chắn bằng 0?

A Lúc nào cũng vậy, vì mỗi điểm là hàng xóm gần nhất của chính nó

B Khi không có hai điểm train trùng input mà khác nhãn

C Khi nhiễu nhỏ

D Khi đã chuẩn hóa feature

CÂU 5

Một hàm của scikit-learn báo điểm `neg_mean_squared_error` bằng -0.12. MSE bằng bao nhiêu?

A 0.12

B -0.12

C 8.33

D Không đổi được

CÂU 6

Trên cùng một cách chia, kNN cho MSE trên train và validation như sau. $k = 1$: 0 và 0.48. $k = 5$: 0.07 và 0.21. $k = 25$: 0.25 và 0.29. Chỉ chọn trong ba giá trị này thì chọn k nào?

A $k = 1$, vì lỗi train bằng 0

B $k = 25$, vì train và validation gần nhau nhất

C Lấy trung bình ba giá trị, tức khoảng $k = 10$

D $k = 5$, vì MSE trên validation nhỏ nhất

PHỎNG THEO CMU 10-601 · Practice Problems, Exam 1 · Q3.2

CÂU 7

Khi tăng độ phức tạp của mô hình, chẳng hạn tăng bậc đa thức trong hồi quy, bias và variance thường thay đổi thế nào?

A Cả hai cùng tăng

B Cả hai cùng giảm

C Cả hai không đổi

D Bias giảm, variance tăng

E Bias tăng, variance giảm

ĐỀ THI USA-NA-AIO 2026 · Vòng 1 · Part 1.3

CÂU 8

Giữ nguyên validation, tăng train từ 100 lên 400 rồi 1600 mẫu. MSE trên train là 0.03, 0.07, 0.10. MSE trên validation là 0.50, 0.30, 0.18. Nếu còn kinh phí, ta nên làm gì?

A Giảm train về 100 mẫu để MSE train xuống 0.03

B Thu thêm mẫu cùng loại, tiếp tục theo dõi validation

C Kết luận thêm dữ liệu có hại, vì MSE train tăng

D Chọn lại validation cho hai lỗi gần nhau nhất

PHỎNG THEO CMU 10-601 · Practice Problems, Exam 1 · Q3.1

CÂU 9

Một đội chia ngẫu nhiên train và validation 50 lần, rồi chỉ báo MSE nhỏ nhất trong 50 lần ấy. Làm vậy sai ở đâu?

A Số mẫu độc lập bị tăng lên 50 lần

B Không sai, thử 50 lần thì số nhỏ nhất là công bằng

C Đổi seed nên không chạy lại ra cùng kết quả được

D Chọn lần chia may nhất, nên con số quá lạc quan

CÂU 10

Mô hình đạt điểm tốt khi chia ngẫu nhiên từng hàng. Có chắc nó cũng đoán tốt cho một trường chưa có trong dữ liệu không?

A Có, nếu train đủ lớn và đa dạng

B Có, nếu MSE train bằng 0 ở mọi trường

C Không, phải để riêng trọn vài trường ra mà kiểm

D Có, nếu chia nhiều lần hơn

Đọc thêm

- Dive into Deep Learning — Generalization. Đọc để nối empirical risk, nhiễu và sự khác nhau giữa fitting với generalization.
- James và cộng sự — An Introduction to Statistical Learning, chương 2 và 5. Đọc trực giác bias–variance và cách ước lượng lỗi dự đoán.
- scikit-learn — Validation curves và learning curves. Xem đúng trục của từng loại đường và quy ước score.